

Capítulo 10: Tipos Avanzados de Scala 3: Estructura y Flexibilidad

1. Explicación Teórica: Seguridad y Ergonomía del Tipado

Scala 3 (Proyecto Dotty) eleva la sofisticación de su sistema de tipos, introduciendo construcciones que anteriormente eran complejas o inaccesibles, con el doble objetivo de **mejorar la ergonomía del desarrollador y la solidez (soundness) del lenguaje**. Los Tipos Avanzados permiten modelar relaciones complejas de datos y comportamientos con una precisión que reduce la necesidad de abstracciones en tiempo de ejecución, llevando la lógica a la fase de compilación.

A. Tipos de Unión (*Union Types*: `A | B`)

El tipo de unión (`A | B`) se define como un valor que puede ser **A o B**. En esencia, permite declarar que una variable puede contener uno de varios tipos posibles. Esto es invaluable para diseñar APIs flexibles que aceptan diferentes formas de entrada (polimorfismo de parámetros) sin recurrir al tipo base genérico y poco seguro `Any`. El compilador obliga al desarrollador a manejar explícitamente todos los casos posibles de la unión, lo que aumenta la seguridad.

B. Tipos de Intersección (*Intersection Types*: `A & B`)

El tipo de intersección (`A & B`) representa un valor que es **A y B al mismo tiempo**. Esto significa que el valor posee todas las capacidades (métodos y campos) de ambos tipos. Este concepto es la forma moderna en Scala 3 de expresar la composición de múltiples *traits* (comportamientos), reemplazando la sintaxis `A with B` de Scala 2. Se utiliza principalmente para componer contratos de API y comportamientos modulares.

C. Tipos Opacos (*Opaque Types*)

Los Tipos Opacos (`opaque type`) abordan un problema de rendimiento de larga data en Scala y Java: la necesidad de usar *wrapper classes* (clases envoltorio) para ganar seguridad de tipos. Al declarar un tipo como opaco, se le indica al compilador que, en **tiempo de compilación**, trate `NewType` como un tipo distinto a `UnderlyingType` (ej. `String`), pero que, en **tiempo de ejecución**, `NewType` se compile como el tipo subyacente. Esto logra una **abstracción de costo cero en runtime**, ideal para evitar la sobrecarga de memoria asociada con la creación de miles o millones de *case classes* simples.

2. Sintaxis y Ejemplos de Código

A. Tipos de Unión: `A | B` (O bien)

Sintaxis: TipoA | TipoB

Ejemplo realista: Procesar una solicitud de ID que puede venir como `String` (ej. un UUID) o como `Int` (ej. un ID autoincremental).

```
1 def procesarBusqueda(id: String | Int): Unit =
2   id match
3     // El compilador garantiza que 's' es String aquí (Flow Typing)
4     case s: String => println(s"Buscando UUID: $s")
5
6     // El compilador garantiza que 'n' es Int aquí
7     case n: Int    => println(s"Buscando ID numérico: $n")
8
9   // No se necesita el caso '_' o default si se cubren todas las variantes.
10  // El compilador sabe que 'id' solo puede ser String o Int.
```

Nota: En lenguajes orientados a objetos tradicionales, esto requeriría usar una clase base común irrelevante o usar el tipo `Any`, perdiendo la seguridad de tipos.

B. Tipos de Intersección: A & B (Y también)

Sintaxis: TipoA & TipoB

Ejemplo realista: Definir un servicio que debe cumplir dos contratos: ser un `Logger` (para registrar) y un `Cache` (para almacenar) simultáneamente.

```
1 trait Logger:
2   def log(msg: String): Unit
3
4 trait Cache:
5   def put(key: String, value: String): Unit
6
7 // Define una única referencia que cumple ambos contratos
8 // Nota: En Scala 2 sería 'val servicio: Logger with Cache'
9 val servicio: Logger & Cache = new Logger with Cache:
10   def log(msg: String): Unit = println(s"[LOG] $msg")
11   def put(key: String, value: String): Unit = println(s"[CACHE] $key -> $value")
12
13 // Puedes llamar a los métodos de ambos traits en la misma referencia:
14 servicio.log("Inicio de operación")
15 servicio.put("session_123", "data")
```

C. Tipos Opacos: opaque type

Sintaxis: `opaque type NewName = UnderlyingType`

Ejemplo realista: Crear un tipo seguro para `UserID` que es internamente un `Long` (eficiente) pero que el compilador no permite mezclar con un `IDDeSesión` (también `Long`).

```
1  object DominioSeguridad:
2    // 1. Declaración opaca (costo cero en runtime)
3    // Fuera de este objeto, UserID es un tipo nuevo. Dentro, es un Long.
4    opaque type UserID = Long
5    opaque type SessionID = Long
6
7    // 2. Método de Extensión para añadir API a UserID
8    extension (id: UserID)
9      def isValid: Boolean = id > 0L // Accede al Long subyacente
10
11    // 3. Método de conversión (Factory) para crear UserID desde Long
12    def createUserID(raw: Long): UserID = raw
13
14    // Uso externo:
15    val rawId = 987654321L
16
17    // Error de compilación: rawId es Long, se requiere UserID
18    // val userId: DominioSeguridad.UserID = rawId
19
20    // Correcto: se usa el constructor de fábrica
21    val userId = DominioSeguridad.createUserID(rawId)
22
23    // val sessionId: DominioSeguridad.SessionID = 111L // Error: Invalido
    // fuera del objeto
24
25    // Error de compilación: No se permite comparar tipos diferentes aunque
    // ambos sean Long por debajo
26    // if (userId == 111L) ...
```

3. Comparativa con Java y Python

Aspecto	Java (o POO tradicional)	Python (Dinámico)	Scala 3 (Idiomático)
Seguridad del Estado Primitivo	Requiere <i>wrapper classes</i> (<code>new UserID(id)</code>) que crean <i>overhead</i> en el <i>heap</i> .	No hay seguridad de tipos estática; un ID es solo un <code>int</code> o <code>str</code> .	<code>opaque type</code> . Garantiza seguridad de tipos en compilación con costo de rendimiento nulo en <i>runtime</i> .

Aspecto	Java (o POO tradicional)	Python (Dinámico)	Scala 3 (Idiomático)
Composición de Contratos	Herencia de una sola clase y múltiples <i>interfaces</i> .	Se utiliza la herencia múltiple de clases (<i>mixins</i>).	<code>A & B</code> . Combina la funcionalidad de múltiples <i>traits</i> de forma puramente compositiva.
Flexibilidad de Entrada (O)	Se requiere que los tipos hereden de una superclase común o el uso de <code>Object</code> / <code>Any</code> , reduciendo la precisión.	Tipado dinámico (<code>Union</code> en versiones recientes de Python), pero sin chequeo estático estricto.	<code>A \ B</code> . Permite el polimorfismo de tipos sin jerarquía, con verificación de exhaustividad en el <code>match</code> .

4. Best Practices (*The Scala Way*)

El uso de los tipos avanzados de Scala 3 es un marcador de código moderno y de alto nivel, que busca la **seguridad de tipos en compilación** sin comprometer la **eficiencia en ejecución**.

- Tipado Estricto de Datos Primitivos (`opaque type`):** Para valores que deben ser tratados como tipos únicos (ej. claves de API, IDs, tokens) pero que se basan en tipos primitivos (`String` , `Int` , `Long`), utiliza siempre `opaque type` dentro de un *object* contenedor. Esto te da la seguridad de que no mezclarás accidentalmente un `Password` con un `UserName` (ambos `String`) sin incurrir en el *overhead* de memoria de envolverlos en *case classes*.
- Exposición de API para Tipos Opacos (`extension`):** Dado que el tipo opaco oculta los métodos del tipo subyacente (ej. los métodos de `String`), debes usar `extension methods` dentro del mismo *object* contenedor para proporcionar una API segura y controlada al tipo opaco.
- Composición de Comportamiento (`A & B`):** Evita la herencia de clase rígida (`class extends B`). Para combinar capacidades, utiliza la **intersección de *traits*** (`Logger & Cache`). Esto permite que una única referencia cumpla con múltiples contratos de comportamiento de forma transparente.
- APIs Flexibles y Explícitas (`A | B`):** Utiliza `Union Types` para simplificar la firma de métodos que aceptan entradas variadas. Asegúrate de procesar siempre el resultado de una unión mediante `match expression` para aprovechar el chequeo de exhaustividad del compilador, forzándote a manejar todas las posibilidades.

El sistema de tipos avanzado de Scala 3 actúa como un **microscopio de precisión** que permite al desarrollador inspeccionar y controlar las formas exactas de los datos y las

relaciones, convirtiendo lo que en Java sería un error de *runtime* (`NullPointerException` o lógica incorrecta) en una advertencia en tiempo de compilación.